

INTERNLINK

Full-Stack PHP Job & Internship Platform

Complete Project Development Guide

Everything you need to plan, build, test and present the project.

HTML • CSS • JavaScript • PHP • MySQL • PDO • Git/GitHub • XAMPP

1. Project Overview

InternLink connects students looking for internships/jobs with companies looking for candidates. Students create profiles, search opportunities, apply and track applications. Companies create profiles, post jobs and manage applicants. Administrators control the platform.

Main goals

- Practice PHP backend development with a real-world application.
- Learn relational database design and SQL.
- Build authentication and role-based access control.
- Implement CRUD operations, search, filtering, file uploads and sessions.
- Create separate student, company and admin experiences.
- Finish with a portfolio project that can be demonstrated to recruiters.

2. Core Features

Student side

- Home and job search
- Registration/login/logout
- Student profile
- CV upload
- Browse/search/filter jobs
- Job details
- Apply with cover letter and CV
- Application tracking
- Saved jobs
- Notifications

Company side

- Company registration/login
- Company profile
- Create/edit/delete job posts
- View applicants
- View candidate profiles
- Download CVs
- Shortlist/accept/reject applications

Admin dashboard

- Dashboard statistics
- Manage students
- Manage companies

- Approve/reject companies
- Approve/reject job posts
- Manage applications
- Suspend/delete users
- Reports and recent activity

3. Recommended Pages

Area	Pages
Public	Home, Jobs, Job Details, About, Contact, Login, Register
Student	Dashboard, Profile, Edit Profile, Saved Jobs, Applications, Notifications
Company	Dashboard, Company Profile, Post Job, Manage Jobs, Applicants, Notifications
Admin	Dashboard, Users, Companies, Jobs, Applications, Reports, Settings

4. Database Design

Database name: **internlink_db**

users

```
id INT PRIMARY KEY AUTO_INCREMENT
name VARCHAR(100)
email VARCHAR(150) UNIQUE
password VARCHAR(255)
role ENUM('student','company','admin')
status ENUM('active','suspended') DEFAULT 'active'
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

student_profiles

```
id INT PRIMARY KEY AUTO_INCREMENT
user_id INT UNIQUE
university VARCHAR(150)
course VARCHAR(150)
level VARCHAR(50)
phone VARCHAR(30)
bio TEXT
profile_picture VARCHAR(255)
cv VARCHAR(255)
linkedin VARCHAR(255)
github VARCHAR(255)
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

company_profiles

```
id INT PRIMARY KEY AUTO_INCREMENT
user_id INT UNIQUE
company_name VARCHAR(150)
logo VARCHAR(255)
description TEXT
industry VARCHAR(100)
location VARCHAR(150)
website VARCHAR(255)
phone VARCHAR(30)
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

jobs

```

id INT PRIMARY KEY AUTO_INCREMENT
company_id INT
title VARCHAR(200)
description TEXT
requirements TEXT
location VARCHAR(150)
job_type VARCHAR(50)
salary VARCHAR(100)
deadline DATE
status ENUM('pending','approved','rejected','closed')
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

```

applications

```

id INT PRIMARY KEY AUTO_INCREMENT
job_id INT
student_id INT
cover_letter TEXT
cv VARCHAR(255)
status ENUM('pending','shortlisted','accepted','rejected')
applied_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

```

saved_jobs

```

id INT PRIMARY KEY AUTO_INCREMENT
student_id INT
job_id INT
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
UNIQUE(student_id, job_id)

```

skills + student_skills

```

skills: id, name
student_skills: id, student_id, skill_id

```

notifications

```

id INT PRIMARY KEY AUTO_INCREMENT
user_id INT
title VARCHAR(150)
message TEXT
is_read BOOLEAN DEFAULT FALSE
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

```

NOTE: Use foreign keys between related tables. Core relationships: users → profiles; company → jobs; student + job → applications; student + job → saved_jobs; student ↔ skills; users → notifications.

5. Database Relationships

```

users
■■■ student_profiles
■■■ company_profiles
■■■ admin access

company_profiles
■■■< jobs

student_profiles
■■■< applications >■■ jobs

student_profiles
■■■< saved_jobs >■■ jobs

student_profiles
■■■< student_skills >■■ skills

users

```

```
■■■< notifications
```

6. Suggested Project Folder Structure

```
internlink/  
■■■ index.php  
■■■ login.php  
■■■ register.php  
■■■ logout.php  
■■■ jobs.php  
■■■ job-details.php  
■■■ apply.php  
■■■ config/database.php  
■■■ includes/  
■ ■■■ header.php  
■ ■■■ footer.php  
■ ■■■ auth.php  
■ ■■■ functions.php  
■■■ student/  
■■■ company/  
■■■ admin/  
■■■ uploads/  
■ ■■■ cvs/  
■ ■■■ profiles/  
■ ■■■ logos/  
■■■ assets/  
■ ■■■ css/  
■ ■■■ js/  
■ ■■■ images/  
■■■ database/internlink.sql
```

7. Build Order — Follow This

Phase 1 — Planning

Write requirements, pages, roles and database relationships before coding.

Phase 2 — UI

Build public pages and dashboards with HTML/CSS first.

Phase 3 — Database

Create the database, tables, keys, constraints and test data.

Phase 4 — PHP connection

Create database.php using PDO and test SELECT/INSERT.

Phase 5 — Authentication

Register, login, logout, password_hash/password_verify and sessions.

Phase 6 — Role protection

Students, companies and admins access only their own areas.

Phase 7 — Jobs CRUD

Companies create/read/update/delete jobs; admin approves jobs.

Phase 8 — Applications

Students apply; companies view applications and change status.

Phase 9 — Search/filter

Keyword, location, type and deadline filters.

Phase 10 — Profiles/CVs

Profile editing, CV upload and controlled CV download.

Phase 11 — Saved jobs/notifications

Add saved jobs and notification records.

Phase 12 — Admin/reporting

Statistics, moderation, user management and reports.

Phase 13 — Security/testing

Validation, authorization, SQL injection protection, upload checks and testing.

Phase 14 — Deployment

Production database, hosting and GitHub README.

8. PHP Concepts to Practice

- Variables, arrays, functions and conditionals
- GET vs POST
- include/require
- Sessions and cookies
- PDO and prepared statements
- CRUD operations
- Password hashing
- File uploads
- Validation and sanitization
- Authentication vs authorization
- Role-based access control
- Pagination
- SQL LIKE search
- JOINS and relational queries
- Transactions
- Error handling
- Reusable functions/components

9. SQL Skills to Practice

- CREATE DATABASE / CREATE TABLE

- PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, DEFAULT
- INSERT, SELECT, UPDATE, DELETE
- WHERE, ORDER BY, LIMIT
- LIKE
- INNER JOIN / LEFT JOIN
- COUNT, SUM and GROUP BY
- Indexes
- Foreign-key actions such as ON DELETE where appropriate

10. Authentication Flow

```

REGISTER
↓ Validate input
↓ Check email
↓ password_hash()
↓ INSERT user
↓

LOGIN
↓ Find user by email
↓ password_verify()
↓ $_SESSION['user_id']
↓ $_SESSION['role']
↓ Redirect by role

Protected page
↓ Check session
↓ Check role
↓ Allow / deny

```

NOTE: Never store plain-text passwords. Never trust a role supplied by a form or URL. Determine permissions from the authenticated session/database.

11. Security Checklist

- PDO prepared statements
- password_hash() / password_verify()
- Server-side validation
- htmlspecialchars() when rendering user content
- Authentication + authorization checks
- Prevent duplicate applications
- Secure CV/profile uploads: extension, MIME type, size and safe filename
- Do not allow uploaded PHP files to execute
- Add CSRF protection for important POST actions
- Never commit database passwords/secrets
- Use HTTPS in production
- Use least-privilege database credentials

12. PDO Connection Example

```

$host = 'localhost';
$db = 'internlink_db';
$user = 'root';
$pass = '';

$dsn = "mysql:host=$host;dbname=$db;charset=utf8mb4";

$options = [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
];

$pdo = new PDO($dsn, $user, $pass, $options);
?>

```

13. Prepared Query Example

```

$stmt = $pdo->prepare(
    "SELECT * FROM users WHERE email = ?"
);
$stmt->execute([$email]);
$user = $stmt->fetch();

```

14. CRUD Practice

Create

Company creates a job → validate → INSERT into jobs.

Read

Student opens jobs.php → SELECT approved jobs → display results.

Update

Company edits its own job → UPDATE using both job id and company id.

Delete

Company/admin closes or deletes a job → verify permission first.

15. Dashboard Statistics

```

SELECT COUNT(*) FROM users WHERE role = 'student';

SELECT COUNT(*) FROM jobs WHERE status = 'approved';

SELECT COUNT(*) FROM applications;

SELECT status, COUNT(*)
FROM applications
GROUP BY status;

```

Start with server-rendered statistics. Add JavaScript charts later.

16. Testing Checklist

- Register valid/invalid data
- Duplicate email
- Wrong password

- Protected pages without login
- Admin pages as student
- Create/edit/delete job
- Attempt to edit another company's job
- Apply and confirm application
- Duplicate application
- Search/filter
- Valid/invalid CV upload
- Application status changes
- Responsive phone/tablet/desktop
- Empty database and expired jobs

17. Git/GitHub Workflow

```
git init
git add .
git commit -m "Initial InternLink project"
git branch -M main
git remote add origin YOUR_REPOSITORY_URL
git push -u origin main
```

Use clear commits such as: add authentication, create jobs CRUD, add application system, build admin dashboard.

NOTE: Use .gitignore for secrets, private uploads and unnecessary XAMPP/system files.

18. README Should Include

- Project purpose
- Screenshots
- Features
- Tech stack
- Database structure
- Installation/run instructions
- Demo accounts if safe
- Security features
- Future improvements

19. Course Topic → Project Feature

Course topic	InternLink implementation
PHP forms	Register, login, post job, apply
Sessions	Login state and role access
MySQL	Users, jobs, applications
PDO	Database operations
CRUD	Job/user management

SQL JOIN	Jobs + companies; applications + students
File handling	CV, logo, profile image
JavaScript	Search/filter UI and interactions
HTML/CSS	Pages and responsive dashboards
Security	Hashing, prepared statements, authorization

20. MVP — Build This First

Do not build everything at once. Your first working version should have:

- Student/company registration and login
- Student profile
- Company profile
- Company creates jobs
- Student browses jobs
- Student applies
- Company views applications
- Admin approves jobs/users

After this works end-to-end, add saved jobs, notifications, advanced search, analytics, reviews and other extras.

21. Final Portfolio Target

Your finished project should demonstrate that you can build a complete application from database design to frontend and backend logic.

```

Student → discovers → Job
Student → applies → Job
Company → posts → Job
Company → reviews → Application
Admin → manages → Platform

```

Portfolio description: InternLink is a full-stack job and internship platform built with PHP, MySQL, JavaScript, HTML and CSS. It includes role-based authentication, student and company dashboards, job management, applications, CV uploads, search/filtering and an administrative management system.

22. Your Rule While Building

Do not copy a finished project. Build each feature yourself, debug it and understand why the fix works. Use the project to reinforce whatever you are learning in your PHP course.

Recommended progression: UI → database → connection → authentication → roles → jobs → applications → search → admin → security → testing → deployment.